Table of contents

1 Managing code page conversion

1.1 Introduction

Choosing Git to host enterprise-level software assets implies a migration task of the z/OS artifacts. Whether the source code is under the control of a legacy SCM solution or not, these artifacts are typically stored as members within partitioned data sets (PDSs) with a specific encoding. Depending on locale, country languages, regulations or work habits, the encoding of the source code can differ by various specificities such as accents, locale typography and characters. To support these peculiarities, many different EBCDIC code pages were developed in z/OS and are now widely used.

In the Git world, the standard prevailing encoding is UTF-8, which supports most (if not all) of the characters used in printing worldwide. When migrating artifacts from z/OS to Git, a translation must occur to convert the EBCDIC-encoded content of source files to UTF-8. This process is transparent to the user, whereby the translation is performed by the Rocket Git client, as long as the necessary information on how to perform the conversion is provided. Git uses a specific file called `.gitattributes` to describe and dictate the translation behavior. This page contains details on the encoding that should be used when hosting the artifacts on z/OS or on Git repositories.

This page will highlight some specific migration situations and some cautionary commentary that must be taken into account in order to ensure a successful migration of z/OS artifacts to Git. The purpose of this document is not to cover all the country-related specificities, nor the complex encodings found worldwide. Rather it is aimed at presenting general use cases and how to prevent/circumvent potential pitfalls.

1.2 Understanding the code page translation process

Code page translation has always been a challenge since z/OS is exchanging data with the distributed systems. To completely understand the effect of encoding, it is important to take a step back and analyze the binary content of files. This analysis will help to better visualize the transformation process that is occurring behind the scenes

1.2.1 Looking inside a file

A file itself can be summarized as being a series of bytes, often expressed in the hexadecimal notation (from x'00' to x'FF'). This file can represent binary content which is not meant to be human-readable (like an executable file), or text content. For the latter and from a user standpoint, this series of bytes doesn't mean anything if the content is not associated with a code page, which will link each byte to a printable character (or a control character). Printable characters are the formation of a character set.

The combination of the contents of a file and the code page used defines the human-readable content. That said, the contents of a file will be displayed slightly differently when combined with different code pages. Let's explore a couple of examples using the ISPF Editor with the "HEX VERT" option enabled to show the hexadecimal values of a file.

Consider the following example where a file containing the French sentence "Bonjour à tous !" (which translates to "Hello everyone!") is displayed with two different code pages. The file which contains this sentence was saved using IBM-1147 encoding. When the file is then displayed using a different encoding of IBM-1047, although the contents of the file is the same from a binary standpoint, some characters are displayed in a different way:

- Using the IBM-1147 code page:

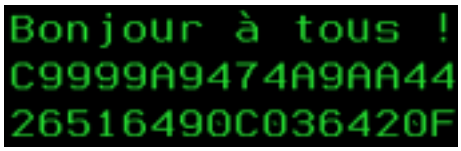

Figure 1: Using the IBM-1147 code page

- Using the IBM-1047 code page:

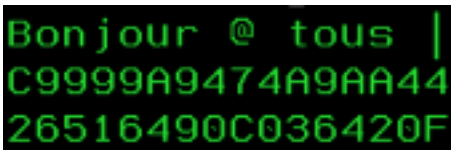

Figure 2: Using the IBM-1047 code page

To be correctly displayed with the IBM-1047 code page, the contents of this file must be transformed. Please note how the hexadecimal codes for the à and the ! characters changed, respectively from x'7C' to x'44' and from x'4F' to x'5A':

- Transforming to the IBM-1047 code page:

```
Bonjour à tous !
C9999A9444A9AA45
265164904036420A
```

Figure 3: Transforming to the IBM-1047 code page

It is important to understand that the code page plays a determining role not only when displaying a file, but also when editing it. To ensure the content of a file is consistent when used by different people, the code page used for editing and displaying will likely be the same for all the users. If Alice edits a file with the IBM-1147 code page and introduces characters (like accents) specific to that code page, then Bob will need to use the IBM-1147 code page to display the content of this file. Otherwise, he may experience the situation depicted earlier, where accents are not displayed correctly. If Bob uses the IBM-1047 code page to edit the file and change the content to replace the special characters by the corresponding accents, then Alice will not be able to display the content of this file like she entered it on her terminal.

Consider this next example. A program file, such as C/370, employs the left ([) and right (]) brackets in some of the language statement constructs. In many US-based legacy applications, the original encoding used to create source files was IBM-037. If these files are then displayed using a different encoding of IBM-1047, again, although the content of the files is the same from a binary standpoint, the brackets are not displayed the correct way:

- Using the IBM-037 code page:

```
void main(int argc, char *argv[])
A98849889489A4898864888945898ABB5
569404195D95301973B038190C1975ABD
```

Figure 4: Using the IBM-1147 code page

- Using the IBM-1047 code page:

```
void main(int argc, char *argvÝ¨)
A98849889489A4898864888945898ABB5
569404195D95301973B038190C1975ABD
```

Figure 5: Using the IBM-1047 code page

To be correctly displayed with the IBM-1047 code page, the contents of this file must be transformed. The hexadecimal codes for the [and the] characters must be changed from x'BA' and x'BB' to x'AD' and x'BD', respectively:

- Transforming to the IBM-1047 code page:

```
void main(int argc, char *argv[])
A98849889489A4898864888945898AAB5
569404195D95301973B038190C1975DDD
```

Figure 6: Transforming to the IBM-1047 code page

Again, it is very important that anyone and everyone who displays or edits the file uses a consistent code page. This can sometimes be a challenge, as the code page to be used is generally specified in the 3270 Emulator (TN3270) client session set-up. Another challenge is trying to determine the original encoding used to create the file.

To summarize, the binary content of a file must be transformed to ensure consistency when displayed with another code page. This process is known as the code page translation and is key when migrating your source from your z/OS platform to a distributed platform, which is using different code pages (and most likely today, UTF-8).

1.2.2 Vocabulary

In this document, we will interchangeably use the coded character set ID (CCSID), its equivalent name, and the code page to designate an encoding. Although not strictly equivalent, they are all often interchanged for the sake of convenience. For instance, the IBM-1047 code page is equivalent to the 1047 coded character set (CCSID) and is usually named IBM-1047. The code page for IBM-037 is equivalent to coded character set (CCSID) 037 and is usually named IBM-037. The CCSID for UTF-8 is 1208 and is linked with many code pages.

To simplify, the code pages will be designated by their common name, for instance IBM-037, IBM-1047, IBM-1147, and UTF-8.

For reference, a list of code pages is available in the Personal Communications documentation site.

1.2.3 The challenge of migrating to UTF-8

If you have ever tried transferring a z/OS data set to a distributed machine running a traditional operating system (like Microsoft Windows or Linux), you probably ended up with similar content when opening the file:

The content is unreadable because the transfer was performed without a code page conversion. This generally happens when the file was transferred as binary, which leaves the content of the file untouched. Most of the transfer utilities offer the capability to transfer the files as binary or as text. In the latter option, code page conversion occurs, and the file should be readable on distributed platforms.

Figure 7: Unreadable file due to transfer without code page conversion

The same conversion must be performed for text files when migrating to Git. Generally, Git uses the UTF-8 code page and, as such, a translation must occur on the content of the transferred files. Likewise, when bringing the files back from Git to z/OS, a backward conversion must be performed to ensure the files are correctly read with the code page in use. Failure in performing this conversion may break the contents of the files, which could become damaged and unreadable. Recovering from this failure is difficult, and the best solution is usually to restart the conversion all over again from the beginning, which often means deleting the files in Git and transferring the same files from z/OS again.

1.2.4 Managing non-printable and non-roundtripable characters

In the traditional EBCDIC code pages, some characters exist to control the different devices themselves or the way text should be displayed. These characters are located in the first 64 positions of the EBCDIC characters tables and can be classified into 2 categories: the non-printable characters and the non-roundtripable characters.

- Non-printable characters: These characters can be translated from EBCDIC to UTF-8 and back to EBCDIC without breaking the content of the file. However, editing files which contain these characters with editing tools on a distributed platform, can cause display issues. Non-printable characters include all EBCDIC characters below 0x40.
- Non-roundtripable characters: The translation is still possible from EBCDIC to UTF-8, but the translation back to EBCDIC is not possible. In this case, the translated file is